

Uppsala University
Department of Information Technology

Assignment 1

Orchestration and Contextualization Using Docker Containers

Course: Data Engineering II

Arnab Kumar Ghosh

April 9, 2026

Contents

1	Task 1: Introduction to Docker Containers and DockerHub	2
1.1	Question 1: Contextualization vs. Orchestration	2
1.2	Question 2: Dockerfile Contents and Key Commands	2
1.2.1	The Dockerfile used in Task 1	2
1.2.2	docker run -it mycontainer/first:v1 bash – Explanation	2
1.2.3	docker ps, docker images, and docker stats	3
1.3	Question 3: docker run vs. docker exec vs. docker build	3
1.4	Question 4: DockerHub and Publishing the Container	3
1.5	Question 5: docker build vs. docker compose	4
1.6	Question 6: Multi-Stage Build	4
2	Task 2: Multi-Container Apache Spark Cluster with docker compose	5
2.1	Dockerfile.spark	5
2.2	docker-compose.yml	5
2.3	Question 1: Explaining the docker compose Configuration	6
2.4	Question 2: Format of the docker compose Configuration File	7
2.5	Question 3: Limitations of docker compose	7
3	Task 3: Orchestration and Contextualization Frameworks	7
3.1	Question 1: Role of Orchestration and Contextualization for Large-Scale Distributed Applications	7
3.2	Question 2: Security Risks and Vulnerability Analysis	8
4	Task 4: Orchestration using Docker Swarm	10
4.1	Objective and Setup	10
4.2	Deployment and Verification	11
4.3	Task Questions	12

1 Task 1: Introduction to Docker Containers and DockerHub

1.1 Question 1: Contextualization vs. Orchestration

Contextualization is the process of configuring a container or virtual machine *before* it runs. This means setting it up with the right software, environment variables, and configuration files so that it is ready to do a specific job. A Dockerfile is a good example: it describes every step needed to prepare the container's environment (installing packages, setting paths, choosing a default command).

Orchestration is the process of managing *multiple* containers that work together as a system. It handles how containers are started, stopped, scaled, networked, and monitored. Tools like Docker Compose, Kubernetes, and Docker Swarm are orchestration tools.

In short: contextualization answers “*what should be inside the container?*”, while orchestration answers “*how do containers work together?*”

1.2 Question 2: Dockerfile Contents and Key Commands

1.2.1 The Dockerfile used in Task 1

Listing 1: Dockerfile

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update
4 RUN apt-get -y upgrade
5 RUN apt-get -y install sl
6
7 ENV PATH="${PATH}:/usr/games/"
8
9 CMD ["echo", "Data Engineering-II."]
```

Line by line explanation:

- `FROM ubuntu:22.04` – Sets the base image. Every instruction below builds on top of Ubuntu 22.04. This is the starting point of the container.
- `RUN apt-get update` – Refreshes the list of available packages from Ubuntu's repositories.
- `RUN apt-get -y upgrade` – Upgrades all installed packages to their latest version. The `-y` flag automatically answers “yes” to confirmation prompts.
- `RUN apt-get -y install sl` – Installs the `sl` program (Steam Locomotive), a command-line animation.
- `ENV PATH="${PATH}:/usr/games/` – Adds `/usr/games` to the system `PATH` so the `sl` command can be found without typing its full path.
- `CMD ["echo", "Data Engineering-II."]` – The default command that runs when the container starts in batch mode. It simply prints the text to the terminal.

1.2.2 `docker run -it mycontainer/first:v1 bash` – Explanation

This command creates and starts a new container from the `mycontainer/first:v1` image and opens an interactive terminal:

- `-i` (interactive): Keeps the standard input open so we can type commands.

- `-t (tty)`: Creates a pseudo-terminal inside the container, giving us a proper shell experience.
- `bash`: Overrides the default `CMD` and starts a Bash shell instead, so we can explore the container manually.

1.2.3 docker ps, docker images, and docker stats

Listing 2: Outputs

```
docker ps
CONTAINER ID   IMAGE                                COMMAND      CREATED        STATUS        PORTS   NAMES
ada47961b48d   mycontainer/first:v1               "bash"      44 hours ago   Up 44 hours   -       gracious_wilbur

docker images

REPOSITORY      TAG    IMAGE ID      CREATED        SIZE
mycontainer/first  v1     24410324eeee  44 hours ago   145MB

docker stats
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %    NET I/O      BLOCK
I/O  PIDS
ada47961b48d   gracious_wilbur     0.00%    1.012MiB / 7.653GiB  0.01%    1.05kB / 0B   0B / 0
B      1
```

- `docker ps`: Shows which containers are currently running.
- `docker images`: Lists all Docker images on the system, including their name, tag, ID, creation time, and disk size.
- `docker stats`: Displays a live stream of resource usage statistics (CPU, memory, network I/O) for running containers. Useful for spotting performance bottlenecks.

1.3 Question 3: docker run vs. docker exec vs. docker build

- `docker build`: Reads a Dockerfile and creates a new Docker *image* from it. Nothing is running yet. This just prepares the image blueprint. Example: `docker build -t mycontainer/first:v1 .`
- `docker run`: Takes an existing image and creates a brand-new *container* from it, then starts it. Each call to `docker run` creates a separate container. Example: `docker run mycontainer/first:v1`
- `docker exec`: Runs a command inside a container that is *already running*. It does not create a new container. Example: `docker exec -it my_running_container bash`

A simple mental model: **build** makes the image, **run** starts a container from the image, and **exec** enters a container that is already running.

1.4 Question 4: DockerHub and Publishing the Container

DockerHub (<https://hub.docker.com>) is a cloud-hosted registry where Docker images can be stored, shared, and downloaded. It plays the same role for containers that GitHub plays for source code.

Benefits of DockerHub:

- Images can be shared with teammates or the public.
- It serves as a central repository, so there is no need to transfer image files manually.
- It hosts thousands of official and community images (e.g., Ubuntu, Nginx, PostgreSQL) that can be used as base images.
- It supports version tagging, so different versions of the same image can coexist.

Steps to push the container:

```
# Tagging the local image with my DockerHub username
docker tag mycontainer/first:v1 bdtiger/de-2-a1:v1

# Log in to DockerHub
docker login

# Push the image
docker push bdtiger/de-2-a1:v1
```

The publicly available container is hosted at:

<https://hub.docker.com/r/bdtiger/de-2-a1>

1.5 Question 5: docker build vs. docker compose

- `docker build` is a command that builds a *single* image from a Dockerfile. It has no knowledge of other containers, networks, or volumes.
- `docker compose` manages *multiple containers* together. We describe all services, networks, and volumes in a `docker-compose.yml` file, and then bring the whole application up with one command: `docker compose up`.

In practice, `docker compose` often calls `docker build` internally to build each service's image before starting the containers.

1.6 Question 6: Multi-Stage Build

A **multi-stage build** uses multiple `FROM` statements in a single Dockerfile. Each stage can have its own base image and purpose. In the final stage, we copy only the binaries we actually need and leave behind all the build tools.

Listing 3: Dockerfile.multistage

```
1 # Stage 1: Builder -- compile sl from source
2 FROM ubuntu:22.04 AS builder
3
4 RUN apt-get update
5 RUN apt-get install -y build-essential
6 RUN apt-get install -y libncurses5-dev
7 RUN apt-get install -y git
8
9 RUN git clone https://github.com/mtoyoda/sl.git /build/sl
10
11 WORKDIR /build/sl
12 RUN make
13
14 # Stage 2: Runtime -- copy only the binary
15 FROM ubuntu:22.04
16
17 RUN apt-get update
```

```

18 RUN apt-get install -y libncurses5
19 RUN rm -rf /var/lib/apt/lists/*
20
21 COPY --from=builder /build/sl/sl /usr/local/bin/sl
22
23 ENV PATH="${PATH}:/usr/local/bin/"
24
25 CMD ["echo", "Data Engineering-II."]

```

Benefits of the multi-stage approach:

- **Smaller final image:** Build tools (gcc, make, git) are present only in the builder stage and are not included in the final image. This can reduce image size by tens or hundreds of megabytes.
- **Improved security:** Fewer packages in the final image means a smaller attack surface. Unused compilers and development libraries cannot be exploited.
- **Cleaner separation of concerns:** The build environment and the runtime environment are kept separate, making the Dockerfile easier to understand and maintain.

2 Task 2: Multi-Container Apache Spark Cluster with docker compose

2.1 Dockerfile.spark

The following Dockerfile is used as the base image for both the master and worker containers:

Listing 4: Dockerfile.spark

```

1 FROM ubuntu:22.04
2 RUN apt-get update
3 RUN apt-get -y upgrade
4 RUN apt install -y openjdk-11-jre-headless
5 RUN apt install -y scala
6 RUN apt install -y wget
7 RUN apt install -y screen
8 RUN wget
   https://archive.apache.org/dist/spark/spark-2.4.3/spark-2.4.3-bin-hadoop2.7.tgz
9 RUN tar xvf spark-2.4.3-bin-hadoop2.7.tgz
10 RUN mv spark-2.4.3-bin-hadoop2.7/ /usr/local/spark
11 ENV SPARK_HOME="/usr/local/spark"
12 ENV PATH="${PATH}:${SPARK_HOME}/bin"
13 ENV SPARK_NO_DAEMONIZE="true"
14 RUN sleep 5
15 CMD ["sh", "-c", "screen -d -m ${SPARK_HOME}/sbin/start-master.sh ;
   ${SPARK_HOME}/sbin/start-slave.sh spark://sparkmaster:7077 && tail -f /dev/null"]

```

2.2 docker-compose.yml

Listing 5: docker-compose.yml

```

1 version: "3.8"
2
3 services:
4   spark-master:
5     build:
6       context: .
7       dockerfile: Dockerfile.spark

```

```

8   hostname: sparkmaster
9   container_name: spark-master
10  ports:
11    - "8080:8080"    # Spark Web UI
12    - "7077:7077"    # Spark master port
13  command: >
14    sh -c "${SPARK_HOME}/sbin/start-master.sh && tail -f /dev/null"
15  networks:
16    - spark-network
17
18  spark-worker:
19    build:
20      context: .
21      dockerfile: Dockerfile.spark
22    container_name: spark-worker
23    depends_on:
24      - spark-master
25    command: >
26      sh -c "${SPARK_HOME}/sbin/start-slave.sh spark://sparkmaster:7077 && tail -f /
dev/null"
27    networks:
28      - spark-network
29
30  networks:
31    spark-network:
32      driver: bridge

```

2.3 Question 1: Explaining the docker compose Configuration

The configuration file defines two services that together form a Spark cluster:

- **spark-master:** This service builds the image from `Dockerfile.spark` and starts the Spark master process.
 - `hostname: sparkmaster` – Sets the internal hostname so the worker can reach the master using the DNS name `sparkmaster`.
 - `ports` – Maps host port 8080 (Web UI) and 7077 (Spark protocol) to the container. The Web UI lets us monitor the cluster from a browser at `http://localhost:8080`.
 - `command` – Starts the master process, then runs `tail -f /dev/null` to keep the container alive.
- **spark-worker:** This service builds from the same image and starts a Spark worker process.
 - `depends_on: spark-master` – Docker Compose will start the master container first, before starting the worker.
 - `command` – Starts `start-slave.sh` with the master URL `spark://sparkmaster:7077` so the worker knows where to register.
- **spark-network:** A custom bridge network is created so both containers can reach each other by their service names. Without this, containers on default bridge networks cannot resolve each other's hostnames.

How to start and test the cluster:

```
# Build images and start both containers
```

```
docker compose up

# Login to the master container
docker exec -it spark-master bash

# Run the following command inside the master container to test the cluster
${SPARK_HOME}/bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master spark://sparkmaster:7077 \
${SPARK_HOME}/examples/jars/spark-examples_2.11-2.4.3.jar

# Expected output:
# Pi is roughly 3.14159...
```

2.4 Question 2: Format of the docker compose Configuration File

The docker compose configuration file uses the **YAML** format. YAML is a human-readable data format that uses indentation to show structure.

The top-level keys in a Compose file are:

- **version:** Specifies the Compose file format version. This determines which features are available. For example, version 3.x is required for Swarm mode features.
- **services:** Defines each container to run. Each sub-key is a service name. Common options per service: build, image, ports, environment, depends_on, command, volumes, networks.
- **volumes:** Declares named volumes that can be shared between services or persisted beyond container restarts.
- **networks:** Declares custom networks to isolate or connect services.

The file is typically named `docker-compose.yml` (or `compose.yml` in newer Docker versions). It is read by the `docker compose` command.

2.5 Question 3: Limitations of docker compose

1. **Single-host only:** Docker Compose is designed to run all containers on *one* machine. It cannot spread containers across multiple physical or virtual machines.
2. **No automatic scaling:** We cannot instruct Compose to automatically add more workers when the load increases. We must manually scale with `docker compose up -scale`.
3. **Limited fault tolerance:** If a container crashes, Compose can restart it (with the restart policy), but it has no advanced health-checking or re-scheduling across hosts.

3 Task 3: Orchestration and Contextualization Frameworks

3.1 Question 1: Role of Orchestration and Contextualization for Large-Scale Distributed Applications

Modern large-scale distributed applications are made up of many independent services that must work together reliably. Running hundreds of containers by hand is impossible. Therefore, we rely on **contextualization** to prepare each container with the right libraries and dependencies, and **orchestration** to manage all containers as a coordinated system.

Benefits of orchestration: A single host can fail, traffic can spike unexpectedly, and software or library updates must be rolled out without downtime. An orchestration framework automates these concerns. It starts containers, restarts failed ones, balances traffic, and scales services up or down automatically.

Four Relevant Frameworks:

1. **Docker Compose:** Docker Compose is the simplest orchestration tool. It reads a single YAML file and starts multiple containers on one host. It is easy to use and ideal for development and testing, but it is limited to a single machine.
2. **Kubernetes:** Kubernetes is the industry-standard orchestrator for production workloads. We describe the *desired state* of the system (e.g., “always have three worker replicas running”), and Kubernetes continuously works to achieve that state. It also supports automatic scaling, self-healing (restarts failed containers), and rich access control. However, Kubernetes has a steep learning curve.
3. **Docker Swarm:** Docker Swarm is Docker’s built-in clustering tool. It extends Docker Compose concepts to a multi-node cluster. A Swarm consists of manager nodes and worker nodes. Its key advantage is that if someone already knows Docker Compose, Swarm requires minimal extra knowledge. It is lighter than Kubernetes but also less feature-rich.

Improving the Task 2 Spark Cluster with Kubernetes and Docker Swarm:

The Task 2 Spark cluster runs on a single host using Docker Compose. In a real production environment this has obvious limitations. One node is a single point of failure, and scaling means manually editing the Compose file.

With Docker Swarm: The same Compose file can be deployed as a Swarm stack across multiple VMs. The master and worker services become Swarm services, which can be scaled easily. Swarm also handles restarting failed workers and distributing them across nodes automatically.

With Kubernetes: The Spark master can be deployed as a Deployment with a Service exposing port 7077, and each worker can be a separate Pod managed by a ReplicaSet. Kubernetes’ Horizontal Pod Autoscaler can watch CPU usage and add workers when the cluster is under load. ConfigMaps can inject Spark configuration without rebuilding images. Additionally, Kubernetes offers namespace-level isolation, role-based access control, and integration with cloud auto-scaling groups (e.g., AWS EKS, Google GKE).

In summary, Docker Compose is excellent for local development of the Spark cluster. Docker Swarm provides an easy path to multi-node deployment, and Kubernetes provides the full set of features needed for a production-grade, self-healing, autoscaling Spark environment.

3.2 Question 2: Security Risks and Vulnerability Analysis

Security Risks in the Task 2 Application

The Task 2 multi-container Spark cluster is a good learning environment, but it has several security weaknesses that would be unacceptable in a production system.

1. **Containers run as root.** By default, the Ubuntu-based containers run all processes as the `root` user. If an attacker manages to escape the container, they would have full root access to the host system. The fix is to add a non-privileged user in the Dockerfile and use `USER` instruction to switch to it.

2. **Exposed ports without authentication.** Port 8080 (Spark Web UI) and 7077 (Spark master) are exposed to the host without any authentication. Anyone who can reach the host can submit arbitrary jobs to the Spark cluster. In production, these ports should be protected by a firewall, VPN, or reverse proxy with authentication.
3. **Outdated base image and dependencies.** The image uses Ubuntu 22.04 with Spark 2.4.3 (released in 2019). Older software frequently contains known vulnerabilities (CVEs). Rebuilding the image regularly against an updated base helps address these vulnerabilities.
4. **Downloading binaries over HTTP during build.** The Dockerfile downloads the Spark tarball from an archive URL. Without verifying the checksum, there is a risk of downloading a tampered file. The SHA-512 checksum published by Apache should be verified after every download.
5. **No network segmentation.** Both master and worker share one bridge network, and that network is also reachable from the host. Stricter internal networks and firewall rules would limit the blast radius of a compromised container.

Vulnerability Analysis with Trivy

Trivy is a widely-used open-source container vulnerability scanner by Aqua Security. It can scan Docker images for known CVEs in OS packages and application libraries.

Trivy Scan: `a1-spark-worker:latest`

The `a1-spark-worker:latest` image was scanned using:

```
trivy image a1-spark-worker:latest > trivy-report.txt
```

Key findings:

- The **Ubuntu 22.04** base OS had **44 vulnerabilities**. These are known CVEs in system packages that have not been patched because the image has not been rebuilt against an updated base.
- `htrace-core-3.1.0-incubating.jar` had **51 vulnerabilities** – the highest count of any single file in the image. This is an old distributed tracing library bundled with Hadoop.
- `hadoop-common-2.7.3.jar` had **7 vulnerabilities**. Hadoop 2.7.3 is very old and has many known security issues.
- `commons-compress-1.8.1.jar` had **6 vulnerabilities**, and `avro-1.8.2.jar` had **5 vulnerabilities**.
- Several other Spark JAR dependencies each had 1–3 vulnerabilities, including `guava`, `ivy`, `jackson-core`, `hive-exec`, and `commons-beanutils`.

Trivy Scan: `a1-spark-master:latest`

The `a1-spark-master:latest` image was scanned separately using:

```
trivy image a1-spark-master:latest > trivy-report-spark-master.txt
```

Key findings:

- The **Ubuntu 22.04** base OS had the same **44 vulnerabilities** as the worker image (35 LOW, 9 MEDIUM, 0 HIGH, 0 CRITICAL). This is expected since both images share the same base.
- `spark-2.4.3-yarn-shuffle.jar` had the highest count with **56 vulnerabilities**. This JAR is bundled for YARN integration and is a very old, unpatched version.
- `jackson-databind-2.6.7.1.jar` had **48 vulnerabilities**. The Jackson Databind library has a long history of deserialization CVEs and this version is severely outdated.
- `spark-core_2.11-2.4.3.jar` had **18 vulnerabilities**, directly in the core Spark runtime.
- `mesos-1.4.0-shaded-protobuf.jar` had **9 vulnerabilities** and `snakeyaml-1.15.jar` and `netty-3.9.9.Final.jar` each had **8 vulnerabilities**.
- `log4j-1.2.17.jar` and `commons-compress-1.8.1.jar` each had **6 vulnerabilities**. The Log4j 1.x branch reached end-of-life in 2015 and has several known RCE-related CVEs.
- Several further JARs (`avro-1.8.2.jar`, `xercesImpl-2.9.1.jar`) had **5 vulnerabilities** each, and `libthrift-0.9.3.jar`, `zookeeper-3.4.6.jar`, `snappy-java-1.1.7.3.jar` each had **4 vulnerabilities**.

Comparison with `a1-spark-worker:latest`:

Both images are built from the same `Dockerfile.spark`, so they share identical OS and JAR vulnerabilities. The master image additionally includes the YARN shuffle JAR and a broader set of Spark runtime JARs, resulting in a slightly higher total JAR vulnerability count. The root cause for all findings is the same: Spark 2.4.3 is an end-of-life release whose bundled dependencies have not been updated for several years.

4 Task 4: Orchestration using Docker Swarm

4.1 Objective and Setup

The goal of this task was to move our Apache Spark cluster from Docker Compose to Docker Swarm. This means moving from a simple local setup to a system built for multi-node environments.

First, I started Docker Swarm on my machine. This made my computer the manager node of the cluster:

```
docker swarm init
```

Next, I created a new file `docker-compose-swarm.yml` and adapted the `docker-compose.yml` file to work with Swarm. I removed the `build` commands because Swarm needs images that are already built. I also added `deploy` sections to tell Swarm how many copies of each container to run. Finally, I changed the network type from `bridge` to `overlay` so containers can communicate across the Swarm network.

Listing 6: `docker-compose-swarm.yml`

```
1 version: "3.8"
2
3 services:
4   spark-master:
5     image: a1-spark-master:latest
6     hostname: sparkmaster
```

```

7   container_name: spark-master
8   ports:
9     - "8080:8080"    # Spark Web UI
10    - "7077:7077"    # Spark master port
11   command: >
12     sh -c "${SPARK_HOME}/sbin/start-master.sh && tail -f /dev/null"
13   networks:
14     - spark-network
15   deploy:
16     replicas: 1
17     placement:
18       constraints:
19         - node.role == manager # Keep the master on the manager node
20
21   spark-worker:
22     image: a1-spark-worker:latest
23     container_name: spark-worker
24     depends_on:
25       - spark-master
26     command: >
27       sh -c "${SPARK_HOME}/sbin/start-slave.sh spark://sparkmaster:7077 && tail -f /
28       dev/null"
29     networks:
30       - spark-network
31     deploy:
32       replicas: 2 # Let's spin up 2 workers to demonstrate Swarm scaling!
33       restart_policy:
34         condition: on-failure
35
36   networks:
37     spark-network:
38       driver: overlay

```

4.2 Deployment and Verification

The following Docker images were already built and available locally:

```

arnab@student-208-121 A1 % docker images
a1-spark-master      latest      da38cde6f1e3   45 hours ago   1.17GB
a1-spark-worker      latest      8645cbad8364   45 hours ago   1.17GB
sparkaio/first       v0         b05746296546   45 hours ago   1.17GB
bdtiger/multi-stage  v1         6945a548f2d3   46 hours ago   144MB
mycontainer/first    v1         24410324eeee   3 days ago     145MB
bdtiger/de-2-a1      v1         24410324eeee   3 days ago     145MB

```

Since Swarm cannot build images itself, I used the pre-built Docker image from Task 2. Then, I launched the application on the Swarm cluster:

```
docker stack deploy -c docker-compose-swarm.yml spark_stack
```

To check if the deployment worked, I used the `docker service ls` command and `docker stack services spark_stack` command.

```

docker stack ls
NAME                SERVICES
spark_stack        2

docker stack services spark_stack

```

ID	NAME	MODE	REPLICAS	IMAGE
	PORTS			
5am5w1817nwe	spark_stack_spark-master	replicated	1/1	a1-spark-master:
latest	*:7077->7077/tcp, *:8080->8080/tcp			
mdckdov5n6kg	spark_stack_spark-worker	replicated	2/2	a1-spark-worker:
latest				

4.3 Task Questions

1. What are the key differences between docker-compose and Docker Swarm?

- **Design:** Docker Compose is designed to run containers on a single machine. Docker Swarm is designed to run applications across multiple machines.
- **Building Images:** Compose can build images directly from your code files. Swarm cannot do this. It requires the images to be built beforehand.

2. How does Docker Swarm improve scalability and service management compared to docker-compose?

- **Better Scaling:** Swarm can add more containers across different servers just by updating the replica count in the configuration. Compose is limited to the resources of a single server.
- **Automatic Fixes:** If a container crashes, Swarm detects the failure and starts a new container to replace it. Compose cannot do this on its own.

3. What challenges did you face when migrating from Docker Compose to Docker Swarm?

- **Image Building:** Since Swarm cannot build images, I had to make sure all the necessary images were built and available before deploying. This added an extra step to the workflow.